Zigflow DSL Cheatsheet

Ready Patterns (Showcase / Interview)

```yaml
# Parallel — all branches run, output = array
- parallelFetch:
    fork:
      compete: false
      branches:
        - branchA:
            call: http
            with:
              method: get
              endpoint: https://api.example.com/a
        - branchB:
            call: http
            with:
              method: get
              endpoint: https://api.example.com/b

# Race — first branch wins
- raceFetch:
    fork:
      compete: true
      branches:
        - primary:
            call: http
            with:
              method: get
              endpoint: https://primary.example.com/data
        - fallback:
            call: http
            with:
              method: get
              endpoint: https://fallback.example.com/data

# Wait for human approval
- waitApproval:
    listen:
      to:
        one:
          with:
            id: approve        # matches: temporal workflow signal --name
approve
            type: signal
```

Minimal Workflow

```yaml
document:
  dsl: "1.0.0"
  taskQueue: my-queue
  workflowType: my-workflow
  version: "1.0.0"

do:
  - step:
      set:
        message: hello
```

---

document Block

```yaml
document:
  dsl: "1.0.0"                # required — always "1.0.0"
  taskQueue: my-queue          # required — Temporal Task Queue (case-sensitive)
  workflowType: my-wf          # required — Temporal Workflow Type (case-sensitive)
  version: "1.0.0"             # required — semantic version
  metadata:                    # optional
    activityOptions:
      startToCloseTimeout:
        minutes: 5
      heartbeatTimeout:
        seconds: 30
```

---

do Block

```yaml
do:
  - taskName:                 # name is required — used by flow directives
      <task-definition>
  - anotherTask:
      <task-definition>
```

Task Types

set — assign variables

```yaml
- init:
    set:
      userId: ${ $input.userId }
```

```
      status: pending
      requestId: ${ uuid }
```

---

call — HTTP

```
- getUser:
    call: http
    with:
      method: get
      endpoint: https://api.example.com/users/1
    output:
      as:
        user: ${ . }
```

call — POST with body

```
- createOrder:
    call: http
    with:
      method: post
      endpoint: https://api.example.com/orders
      body:
        userId: ${ $input.userId }
        item: ${ $input.item }
    output:
      as:
        order: ${ . }
```

call — OpenAPI

```
- findPets:
    call: openapi
    with:
      document:
        endpoint: https://petstore.swagger.io/v2/swagger.json
      operationId: findPetsByStatus
      parameters:
        status: available
```

call — gRPC

```
- greet:
    call: grpc
```

```
    with:
      proto:
        endpoint: file://app/greet.proto
      service:
        name: GreeterApi.Greeter
        host: localhost
        port: 5011
      method: SayHello
      arguments:
        name: ${ $input.name }
```

---

fork — parallel (all results)

```
- gatherAll:
    fork:
      compete: false          # all branches run; output = array of results
      branches:
        - fetchA:
            call: http
            with:
              method: get
              endpoint: https://api.example.com/a
        - fetchB:
            call: http
            with:
              method: get
              endpoint: https://api.example.com/b
# output: [{result of fetchA}, {result of fetchB}]
```

fork — race (first wins)

```
- fastest:
    fork:
      compete: true           # first branch to finish wins
      branches:
        - primary:
            call: http
            with:
              method: get
              endpoint: https://primary.example.com/data
        - fallback:
            call: http
            with:
              method: get
              endpoint: https://fallback.example.com/data
# output: result of whichever branch completes first
```

---

for — loop over array

```yaml
- processAll:
    for:
      each: item              # loop variable (default: item)
      in: .items              # jq expression for the array
      at: index               # index variable (default: index)
    do:
      - processOne:
          call: http
          with:
            method: post
            endpoint: https://api.example.com/process
            body:
              id: ${ $item.id }
              pos: ${ $index }
```

for — with while condition

```yaml
- processWhile:
    for:
      each: item
      in: .queue
    while: ${ .continue == true }
    do:
      - handle:
          set:
            processed: ${ $item.id }
```

---

listen — wait for one signal

```yaml
- waitApproval:
    listen:
      to:
        one:
          with:
            id: approve        # matches --name in temporal workflow signal
            type: signal
```

listen — wait for any of multiple events

```yaml
- waitVital:
    listen:
      to:
```

```
      any:
        - with:
            type: com.hospital.vitals.temperature
            data: ${ .temperature > 38 }
        - with:
            type: com.hospital.vitals.bpm
            data: ${ .bpm < 60 }
```

listen — wait for all events

```
- waitAll:
    listen:
      to:
        all:
          - with:
              type: com.example.event.a
          - with:
              type: com.example.event.b
```

---

switch — conditional branching

```
- route:
    switch:
      - isPremium:
          when: .tier == "premium"
          then: fastPath        # jump to named task
      - isBasic:
          when: .tier == "basic"
          then: slowPath
      - default:                # no 'when' = default
          then: rejectRequest

- fastPath:
    call: http
    with:
      method: post
      endpoint: https://api.example.com/premium
    then: end                   # flow directive: end workflow

- rejectRequest:
    raise:
      error:
        type: https://errors.example.com/unauthorized
        status: 403
        title: Access Denied
```

Flow directives for `then`: `continue` · `exit` · `end` · `<task-name>`

try — error handling with retry

```yaml
- safeCall:
    try:
      - fetch:
          call: http
          with:
            method: get
            endpoint: https://unstable.example.com/data
    catch:
      as: error
      retry:
        delay:
          seconds: 2
        backoff:
          exponential: {}      # 2s → 4s → 8s...
        limit:
          attempt:
            count: 3
      do:
        - fallback:
            set:
              result: default_value
```

try — catch without retry

```yaml
- safeCall:
    try:
      - fetch:
          call: http
          with:
            method: get
            endpoint: https://api.example.com/resource
    catch:
      do:
        - setError:
            set:
              err: "Request failed"
```

---

wait — durable timer

```yaml
- pause:
    wait:
      seconds: 30

- longWait:
```

```
    wait:
      hours: 2
      minutes: 30

- isoWait:
    wait: PT1H30M             # ISO 8601 duration string
```

---

run — shell / script / sub-workflow

```
# Shell command
- runShell:
    run:
      shell:
        command: 'echo "Hello ${ $input.name }"'

# JavaScript script
- runScript:
    run:
      script:
        language: js
        code: >
          console.log("done")

# Python script
- runPython:
    run:
      script:
        language: python
        code: >
          print("hello from python")

# Sub-workflow
- runChild:
    run:
      workflow:
        namespace: my-namespace
        name: child-workflow
        version: "1.0.0"
        input:
          userId: ${ $input.userId }
```

---

raise — throw an error

```
- raiseError:
    raise:
      error:
        type: https://errors.example.com/validation
```

```
        status: 400
        title: Invalid Input
        detail: "userId must be a positive integer"
```

Standard error type URIs:

URI suffix	Status
/errors/configuration	400
/errors/validation	400
/errors/expression	400
/errors/authentication	401
/errors/authorization	403
/errors/timeout	408
/errors/communication	500
/errors/runtime	500

---

output.as and export.as

```
# output.as — shapes what flows to the NEXT task only
- fetchUser:
    call: http
    with:
      method: get
      endpoint: https://api.example.com/users/1
    output:
      as:
        user: ${ . }          # next task receives {user: {...}}

# export.as — persists into $context for ALL later tasks
- fetchUser:
    call: http
    with:
      method: get
      endpoint: https://api.example.com/users/1
    export:
      as: "${ $context + {fetchedUser: .} }"  # ALWAYS merge, never replace

# Both together
- fetchAndStore:
    call: http
    with:
      method: get
      endpoint: https://api.example.com/users/1
    output:
```

```
      as:
        user: ${ . }
    export:
      as: "${ $context + {user: .} }"
```

Rule: Use `${ $context + {key: value} }` — never `${ . }` alone in `export.as`.

Runtime Expressions

```
# Workflow input
${ $input.userId }
${ $input.items[0] }

# Previous task output
${ $data.fetchUser.name }
${ $data.createOrder.id }

# Workflow context (persisted across tasks)
${ $context.userId }
${ $context.results }

# Environment variables
${ $env.API_KEY }

# Replay-safe ID and timestamp
${ uuid }
${ timestamp }

# jq filters
${ .users | map(.name) }
${ .users | map(select(.active == true)) }
${ .items | length }
${ if .score >= 50 then "pass" else "fail" end }
${ "Hello, " + .name + "!" }
${ $context + {newKey: .someValue} }
```

Reusable Components (use)

```
use:
  retries:
    standardRetry:
      delay:
        seconds: 2
      backoff:
        exponential: {}
      limit:
        attempt:
```

```
          count: 3

  authentications:
    apiAuth:
      bearer:
        token: ${ $env.API_TOKEN }
    basicAuth:
      basic:
        username: ${ $env.API_USER }
        password: ${ $env.API_PASS }

do:
  - callApi:
      try:
        - fetch:
            call: http
            with:
              method: get
              endpoint:
                uri: https://api.example.com/data
                authentication:
                  use: apiAuth
      catch:
        retry:
          use: standardRetry
```

Task-level Metadata (timeout override)

```
- slowTask:
    metadata:
      timeout: 10m              # overrides document-level activityOptions
    call: http
    with:
      method: post
      endpoint: https://slow-api.example.com/process
```

Full Pattern: init → fetch → signal → route → complete

```
document:
  dsl: "1.0.0"
  taskQueue: order-queue
  workflowType: process-order
  version: "1.0.0"
  metadata:
    activityOptions:
      startToCloseTimeout:
        minutes: 2
```

```yaml
do:
  - init:
      set:
        orderId: ${ uuid }
        status: pending
      export:
        as: "${ $context + {orderId: ${ uuid }, status: \"pending\"} }"

  - fetchProduct:
      try:
        - get:
            call: http
            with:
              method: get
              endpoint: https://api.example.com/products/${
$input.productId }
            output:
              as:
                product: ${ . }
      catch:
        retry:
          delay:
            seconds: 3
          backoff:
            exponential: {}
          limit:
            attempt:
              count: 3

  - waitApproval:
      listen:
        to:
          one:
            with:
              id: approve
              type: signal

  - route:
      switch:
        - approved:
            when: $data.waitApproval.approved == true
            then: complete
        - default:
            then: cancel

  - complete:
      set:
        status: completed
      then: end

  - cancel:
```

```
      set:
        status: cancelled
```